INDEX

SL. NO	TOPICS	SIGN
1.	Display Fibonacci series up to n tems using command line arguments	
2.	Demonstrate single inheritance	
3.	Sort n elements using an array	
4.	Implement constructor overloading by passing different number of parameters of different types.	
5.	Demonstrate string Methods	
6.	Demonstrate Vector Methods	
7.	Demonstrate concept of interface	
8.	Demonstrate concept of creating, accessing and using a package.	
9.	Demonstrate multithreaded programming	
10.	Demonstrate thread Priority	
11.	Create an Applet to draw a human face.	Pending 9/12/20
12.	Demonstrate simple banner applet	
13.	Program to count number of strings, integers and float values through command line arguments.	
14.	Program to accept a message from the keyboard and display the number of words and non-alphabetical characters	
15.	Demonstrate creation of list using an applet.	
16.	Demonstrate concept of Event Handling	
17.	Program to demonstrate different types of fonts	
18.	Create an applet to tokenize the string	
19.	Design a simple calculator using java applet	
20.	Implement static and dynamic stack using	

INDEX

SL. NO	TOPICS	SIGN
	Interface and abstract class	Aparichaa 9/12/25

Lab Program-01

① Display Fibonacci series up to n terms using command line arguments

```
class Fib {
    public static void main(String[] args) {
        if (args.length != 1)
        {
            System.out.println("Dont take
    arguments more than one");
        return;
        }
    int n = Integer.parseInt(args[0]);
    int first = 0, second = 1;
    System.out.println("Fibonacci series up to"
    + n + "terms :");
    for(int i = 1; i <= n; i++) {
    System.out.println(first);
    int next = first + second;
     first = second;
     second = next;
     }
     }
     }
```

Output

Fibonacci series up to 5 terms

0
1
1
2
3
5

Lab Program - 02

② Demonstrate single Inheritance.

```
// Parent class
class Animal {
    void eat() {
    System.out.println("Animals can eat.");
    }
}

//child class (inherits from Animal)
class Dog extends Animal {
    void bark() {
    System.out.println("Dog can bark:");
    }
}

public class SingleInheritanceDemo{
    public static void main (String[] args){
        Dog d = new Dog();
    // using method from parent class
        d.eat();
    // using method from child class
        d.bark();
    }
}
```

Lab Program -03

③ Sort n elements using an array

```
import java.util.Scanner;
public class SortArray{
    public static void main (String[] args) {
     Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements:");
int n= sc.nextInt();
int[] arr = new int[n];
System.out.println("Enter "+n+"elements:");
for (int i=0; i<n;i++) {
arr[i] = sc.nextInt();
 }
for (int i= 0; i<n-1; i++){
for (int j=0; j<n-1-i; j++) {
if (arr[j] >arr[j+1]) {
//swap
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
    }
    }
  }
```

```
System.out.println("Sorted Elements:");
for (int i = 0; i<n; i++){
System.out.println(arr[i]);
    }
 sc.close();
  }
```

Output

```
Enter number of elements  5
Enter 5 elements:
  10
   5
  200
   8
   1.
Sorted elements: 1
                 5
                 8
                 10
                 200
```

Lab Program-04

④ Implement constructor overloading by passing different number of parametrs of different typy

```
class Student {
 String name;
 int age;
 double marks;
 //constructor with no parameters
 Student (){
  name = "Unknown";
  age = 0;
  marks = 0.0;
 }
 //constructor with one parametr (name only)
 Student (String n) {
  name = n;
  age = 0;
  marks = 0.0;
 }
 //constructor with two parametrs (name, age)
 Student (String n, int a) {
  name = n;
```

```
        age = a;
        marks = 0.0;
    }

    // Constructor with three parameters (name, age, marks)
    Student(String n, int a, double m) {
        name = n;
        age = a;
        marks = m;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: "
        + age + ", Marks: " + marks);
    }
}

public class ConstructorOverloadingDemo {
    public static void main(String[] args) {
        // Using different constructors
        Student s1 = new Student();
        Student s2 = new Student("Alice");
        Student s3 = new Student("Bob", 20);
        Student s4 = new Student("Charlie",
                                 22, 85.5);
```

```
// Display Details
S1.display();
S2.display();
S3.display();
S4.display();
}
}
```

Output

```
Name: Unknown, Age: 0, Marks: 0.0
Name: Alice, Age: 0, Marks: 0.0
Name: Bob, Age: 20, Marks: 0.0
Name: Charlie, Age: 22, Marks: 85.5
```

Lab Program-05

⑤ Demonstrate string Methods

```
public class Length {
  public static void main (String[] args) {
    String str1 = "Hello World";
    String str2 = "Java Programming";

  // length()
  System.out.println("Length of str1: "+str1.length());

  // charAt()
  System.out.println("Character at index 4: "+
                     str1.charAt(4));

  // toUpperCase() and toLowerCase()
  System.out.println("Upper case: "+str1.toUpperCase
                     ());
  System.out.println("Lowercase: "+str1.toLowerCase
                     ());

  //trim()
  System.out.println("before trim: "+str2+" ");
  System.out.println("After trim: "+str2.trim()+
                     " ");

  // substring()
  System.out.println("Substring (0,5): "+
```

```
    Str1.substring(0,5));

//equals() and equalsIgnoreCase()
System.out.println("Str1.equals 'Hello world'?"
                    + str1.equals("Hello World"));
System.out.println("str1 equalsIgnoreCase 'hello
 world'?" + str1.equalsIgnoreCase("hello world"));

//contains()
System.out.println("Does str1 contain 'world'?"
 + str1.contains("world"));

// replace()
System.out.println("Replace 'word' with 'Java':"
+ str1.contains("world","Java"));

// Split()
String[] words = str1.split(" ");
System.out.println("splitting str1 :");
 for (String word:words){
 System.out.println(word);
  }
  }
 }
```

Output

```
Length of str1: 11
Character at index 4: O
Uppercase: HELLO WORLD
Lowercase: hello world
Before trim: ' Java Programming'
After trim: 'Java Programming'
Substring (0,5): Hello
Str1 equals 'Hello World'? true
str1 equalsIgnoreCase 'hello world'? true
Does str1 contain 'World'? true
Replace 'World' with 'Java': Hello Java
Splitting str1:
Hello
World.
```

Lab Program - 06

⑥ Demonstrate Vector Methods

```
import java.util.*;
class VectorDemo{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        Vector<String> v = new Vector<>();
        System.out.print("How many fruits? ");
        int n = sc.nextInt();
        sc.nextLine(); // to skip newline
        for(int i=0; i<n; i++){
            System.out.print("Enter fruit name: ");
            String fruit = sc.nextLine();
            v.add(fruit);
        }
        System.out.println("\nFruits : " + v);
        System.out.print("enter a fruit to remove: ");
        String f = sc.nextLine();
        v.remove(f);
        System.out.println("After removing: " + v);
        sc.close();
    }
}
```

Output.

```
How many fruits?
3
Enter fruit name: apple
Enter fruit name : banana
Enter fruit name : orange.
Fruits : [apple, banana, orange]
Enter a fruit to remove: apple
After removing : [banana, orange]
```

Lab Program -07

⑦ Demonstrate concept of interface

```
// Interface
interface Animal {
    void eat();     //abstract method
    void sleep();   //abstract method
}
//class implementing interface
class Dog implements Animal{
    @Override
    public void eat(){
System.out.println("Dog eats bones.");
    }
    @Override
    public void sleep(){
    System.out.println("Dog sleeps in kennel.");
    }
}
// Another class implementing same interface
class Cat implements Animal {
    @Override
    public void eat(){
    System.out.println("Cat eats fish.");
    }
    @Override
    public void sleep() {
```

```
System.out.println("cat sleeps on sofa");
  }
}

public class InterfaceDemo{
    public static void main (String[] args){
Animal a1 = new Dog(); // ~~Inte~~ reference of
                          Interface, object of
                          Dog
Animal a2 = new Cat(); // reference of interface
                          object of cat

a1.eat();
a1.sleep();
a2.eat();
a2.sleep();
  }
}
```

Output

Dog eats bones
Dog sleeps in kennel
cat eats fish
cat sleeps on sofa

Lab Program - 08

(8) Demonstrate concept of creating, accessing and using a Package

```
//File : mypackage/Hello.java
package mypackage;    // This declares the package
public class Hello{
    public void sayHello(){
        System.out.println("Hello from mypackage!");
    }
}
```

```
// File : Main.java
import mypackage.Hello;    // import the package
public class Main{
    public static void main(String[] args){
        Hello obj = new Hello(); // create object of
                                 Hello class
        obj.sayHello();
    }
}
```

Output

Hello from mypackage!

Lab Program - 09

(9) Demonstrate multithreaded programming

```
//Method 1: Extending Thread class
class MyThread extends Thread {
public void run(){
  for(int i=1; i<=5;i++) {
System.out.println(" From myThread: "+i);
try{
Thread.sleep(500); // pause for 0.5 sec
} catch (InterruptedException e) {
  System.out.println(e);
}
}
}
}
//method2: Implementing Runnable Interface
class MyRunnable implements Runnable{
public void run(){
for(int i=1; i<=5;i++){
System.out.println(" From myrunnable: "+i);
try{
Thread.sleep(700); // pause for 0.7 sec
} catch (InterruptedException e) {
System.out.println(e);
```

```
        }
      }
    }
  }
public class Multithreading{
  public static void main (String[] args){
  // using thread class
MyThread t1 = new MyThread();

  //using Runnable Interface
  Thread t2 = new Thread (new MyRunable());

  // start both threads
    t1.start();
    t2. start();

// main thread work
  for(int i=1; i<=5;i++){
System.out.println(" From main Thread " + i );
  try {
Thread.sleep(600);  // pause for 0.6 sec
  }catch (InterruptedException e) {
System.out.println(e);
  }
  }
}
}
```

Output

From MyThread : 1
From Main Thread : 1
From MyRunnable : 1
From MyThread : 2
From MainThread : 2
From MyRunnable : 2
From My Thread : 2
From Main Thread : 2
From MyRunnable : 2
From MyThread : 3
From Main Thread : 4
From My Thread : 5
From MyRunnable : 4
From Main Thread : 5
From MyRunnable : 5

Lab Program -10

(10) Demonstrate thread priority

```
class MyThread extends java.lang.Thread
  {
  MyThread (String name){
    super (name);
    }

public  void run(){
   System.out.println(getName()+" is running with
   priority " + getPriority ());
   }
  }
public class ThreadPriorityDemo {
public static void main (String [] args){
MyThread t1 = new MyThread ("Thread -1");
MyThread t2 = new MyThread ("Thread -2");
MyThread t3 = new MyThread ("Thread -3");
t1.setPriority (java.lang.Thread.MIN_PRIORITY);
t2.setPriority(java.lang.Thread.NORM_PRIORITY);
t3.setPriority(java.lang.Thread.MAX_PRIORITY);
t1.start();
t2.start();
t3.start();
}}
```

Output

```
Thread-3 is running with priority 10
Thread-1 is running with priority 1
Thread-2 is running with priority 5
```

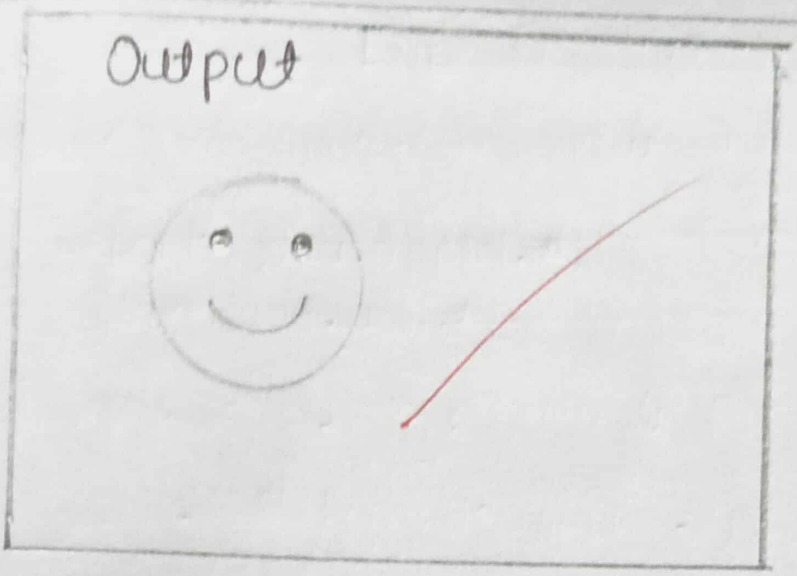
Lab Program - 11

(11) Create an applet to draw a human face.

```
import java.applet.*;
import java.awt.*;

public class face extends Applet{

public void paint(Graphics g){

g.setColor(Color.yellow);          //Face outline
g.fillOval(50,50,200,200);         //Face

g.setColor(color.black);
g.fillOval(100,100,20,20);         //Left eye
g.fillOval(180,100,20,20);         //Right eye

//Nose
g.setColor(Color.red);
g.drawArc(100,130,100,60,180,180);

}
}
```

```
<html>
<body>
<h3>Very Simple Human Face - Java Applet</h3>
<applet code="face.class" width="400" height="400">
</applet>
</body>
</html>
```

Output



Lab Program - 12

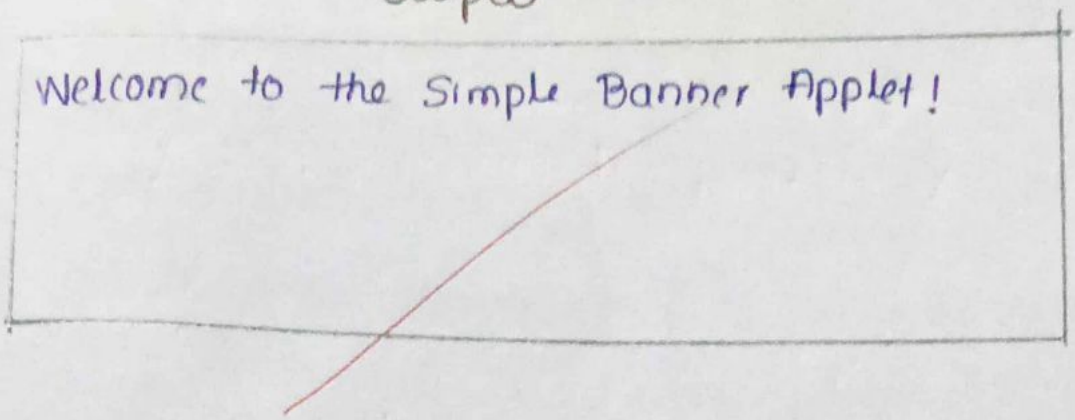
(12) Demonstrate simple banner applet.

```
import java.applet.Applet;
import java.awt.Graphics;

public class BannerApplet extends Applet implements
Runnable{
 String msg = "Welcome to the Simple Banner Applet!";
 Thread t;
 boolean stopFlag;

 public void init(){

  setBackground(Java.awt.Color.white);
  setForeground(Java.awt.Color.blue);

   }
 public void start(){
    t = new Thread(this);
   stopFlag = false;
   t.start();
    }
 public void run(){
  for(;;){
    try {
       repaint();
   Thread.sleep(150);
    msg. msg.substring(1)+msg.charAt(0);//to all
                                        characters
```

```
if (stopFlag)
  break;
 } catch (InterruptedException e) { }
  }
 }
public void stop(){
  stopFlag = true;
   t = null;
  }
public void paint(Graphics g) {
  g.drawString(msg, 20, 30);
  }
 }
```

```
<html>
<body>
  <applet code = "BannerApplet.class" width = "400"
   height = "60">
 </applet>
 </body>
 </html>
```

Output



Lab Program - 13

③ Program to count number of strings, integers and float values through command line arguments

```
public class countArgs {
    Public static void main(String[] args) {
     int countInt=0, countFloat=0, countString=0;
    for (String arg : args) {
    try {
    Integer.parseInt(arg);
    countInt++;
      } catch (NumberFormatException e1) {
    try {
      Float.parseInt(arg);
      countFloat++;
          } catch (NumberFormatException e2) {
      countString++;
        }
      }
    }
System.out.println ("Integers :" + countInt);
System.out.println (" Floats :" + countFloat);
System.out.println("Strings :" + countString);
```

}
}

Output

Chandana 123 , 50

Integer : 2

float : 0

string : 1.

Seen tutor

Lab Program - 14.

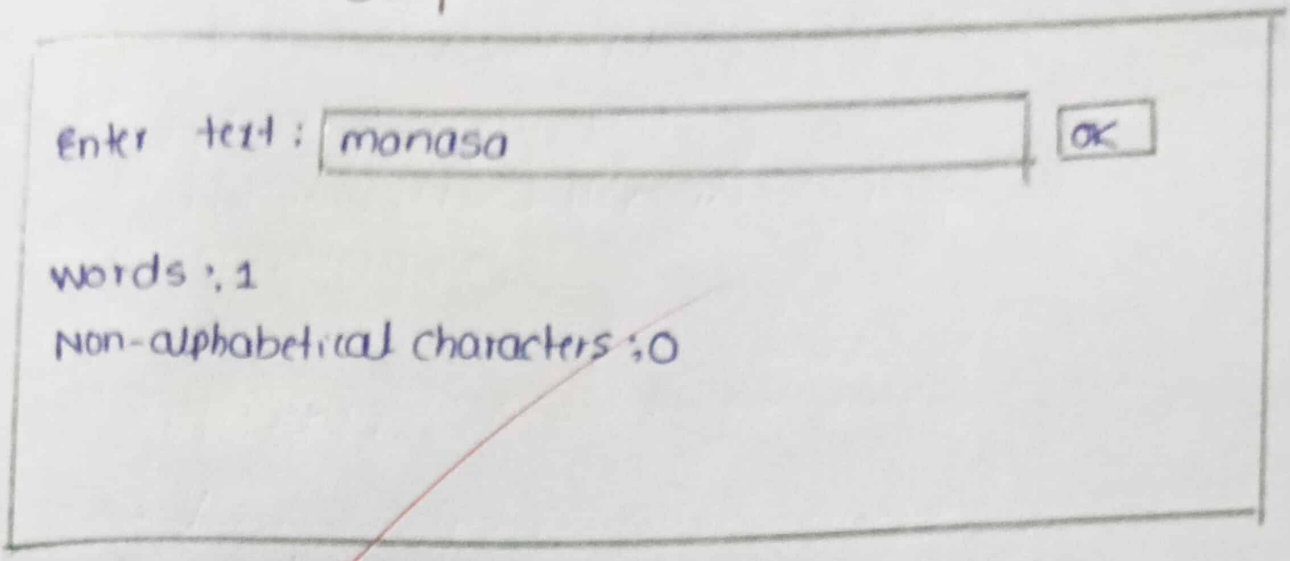
(14) Program to accept a message from the keyboard and display the no. of words and non-alphabetical characters

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class SimpleCountApplet extends Applet implements
ActionListener {

 TextField tf = new TextField(30);
 Button b = new Button("OK");
 int words = 0, nonAlpha = 0;

 public void init(){
     add(new Label("Enter text : "));
      add(tf);
      add(b);
       b.addActionListener(this);
      }

public void actionPerformed(ActionEvent e){
  String s = tf.getText();
   words = 0;
   nonAlpha = 0;

// Count words
  if (s.trim().length()>0 {
     words = s.trim().split("\\s+").length;
     }
```

```
//Counting non-alphabetical characters
for(int i=0; i<s.length(); i++){
    char ch = s.charAt(i);
    if(!Character.isLetter(ch) && ch != ' ')
     nonAlpha++;
    }
    repaint();
    }
public void paint(Graphics g){
    g.drawString("words: " + words, 20, 120);
    g.drawString("Non-alphabetical characters: " +
                    nonAlpha, 20, 140);
        }
    }
```

```
<html>
  <body>
    <applet code="SimpleCountApplet.class" width="400"
     height="200">
    </applet>
  </body>
</html>
```

Output



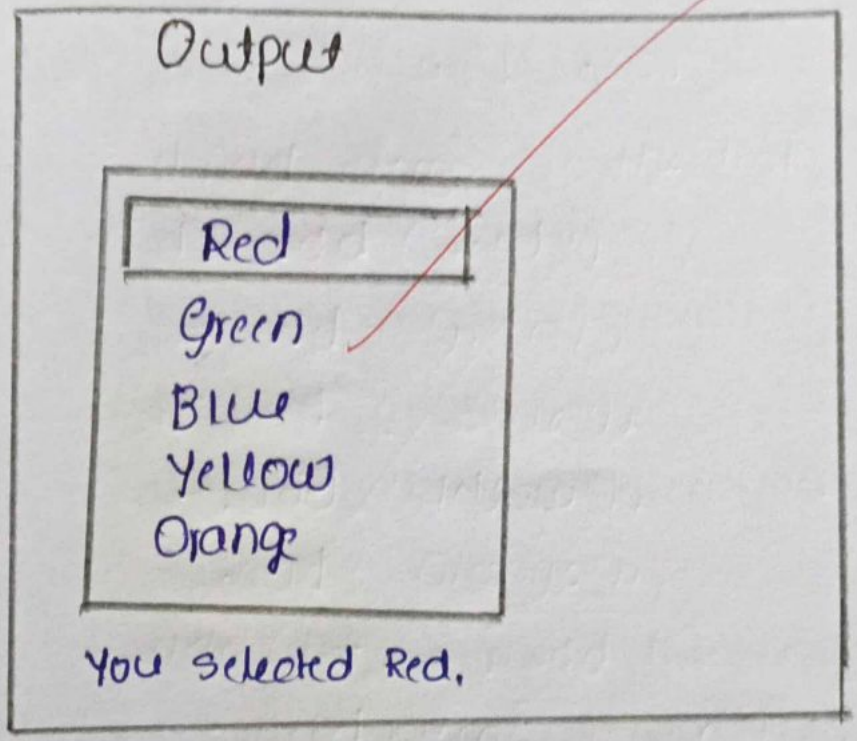
Lab Program -15

(15) Demonstrate creation of list using an applet.

```
import java.applet.Applet;
import java.awt.*;;
import java.awt.event.*;

public class list extends Applet implements ItemListener{
List lst;
String msg = " ";
public void init(){
    //Create a list with 5 visible rows (single-selection)
     lst = new List(5);
    // Add items to the list
     lst.add("Red");
    lst.add("Green");
    lst.add("Blue");
    lst.add("Yellow");
    lst.add("Orange");
    add(lst);  // add list to applet by one
    lst.addItemListener(this);
    }
public void itemStateChanged(ItemEvent e) {
  msg = "You selected: " + lst.getSelectedItem();
  repaint();
  }
public void paint(Graphics g){
```

```
g.drawString(msg, 20, 150);
  }
}
```

```
<html>
<body>
  <applet code = "list.class" width = "300" height = "200">
  </applet>
</body>
</html>
```



Lab Program-16

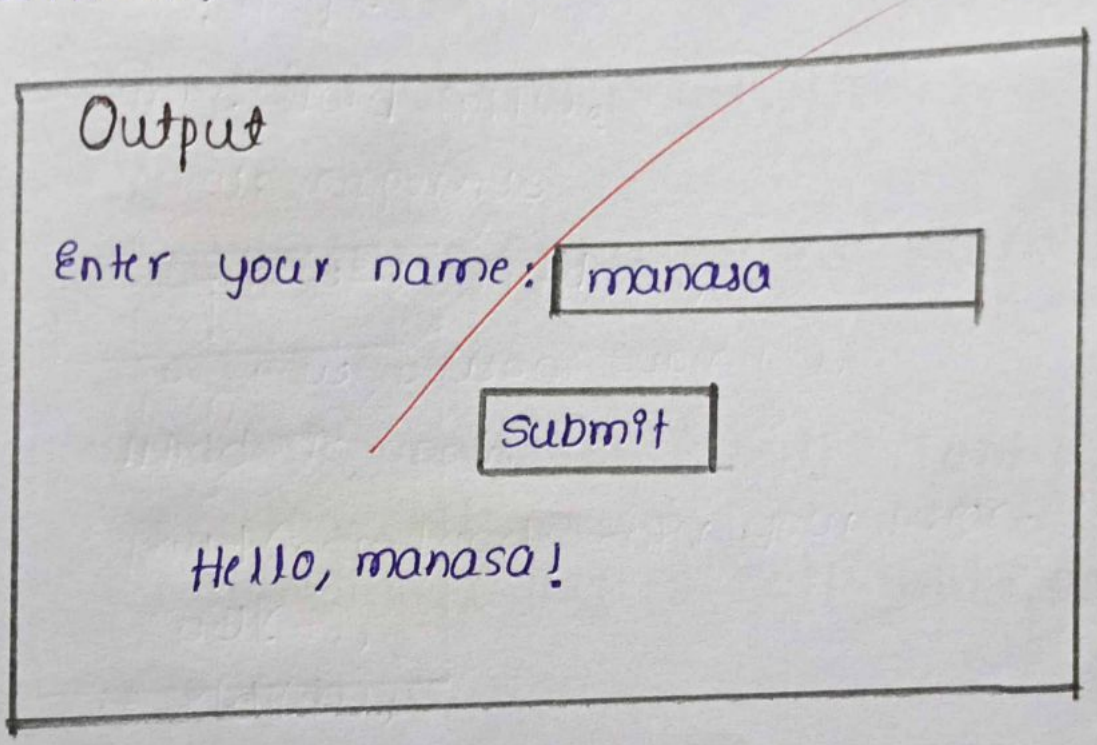
(16) Demonstrate concept of event handling

```
import java.applet.Applet.Applet;
import java.awt.*;
import java.awt.event.*;
public class simpleapplet extends Applet implements
ActionListener {
  Test
  TextField t;
  Button b;
  String msg = " ";
  public void init(){
    //create components
    t= new TextField(15);
    b = new Button("submit");
    //Add to applet
    add(new Label("Enter your name:"));
    add(t);
    add(b);
//Add action listener to button
  b.addActionListener(this);
  }
public void actionPerformed(ActionEvent e) {
  msg = "Hello," + t.getText() + "!";
  repaint();
  }
```

```
public void paint (Graphics g) {
    g.drawString (msg, 50, 100);
    }
    }
```

```
<html>
<body>
<applet code = "simpleapplet.class" width = "300"
 height = "150" > </applet >
</body>
</html>
```

Output



Lab Program - 17

(17) Program to demonstrate different types of fonts

```
import java.applet.Applet;
import java.awt.*;

public class font extends Applet{

    public class font extends Applet{

        public void paint(Graphics g){

    //Font : 1 : Plain

    Font f1 = new Font("Serif", Font.PLAIN, 20);
     g.setFont(f1);
     g.drawString("This is Serif plain", 20, 40);

    //Font 2: Bold

    Font f2 = new Font("Serif", Font.BOLD, 20);
     g.setFont(f2);
     g.drawString("This is Serif Bold", 20, 80);

    //Font 3: Italic

    Font f3 = new Font("Serif", Font.ITALIC, 20);
     g.setFont(f3);
     g.drawString("This is Serif Italic", 20, 120);

    //Font 4: Monospaced

    Font f4 = new Font("Monospaced", Font.BOLD, 22);
     g.setFont(f4);
     g.drawString("This is SansSerif", 20, 200);
```

```
}
}
```

```
<html>
<body>
  <applet code = "font.class" width = "400"
  height = "250" > </applet>
</body>
</html>
```

Output

This is Serif Plain

This is Serif Bold

This is Serif Italic

This is Monospaced

This is SansSerif

Lab Program -18

(18) Create an applet to tokenize the string

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
import java.util.StringTokenizer;

public class TokenizeApplet extends Applet implements
ActionListener {

  TextFiud tf;
  Button b;
  String result = " ";

  public void init(){
     tf = new TextField(30);
     b = new Button("Tokenize");

  add(new Label("Enter a sentence:"));
  add(tf);
  add(b);
  b.addActionListener(this);
 }
public void actionPerformed(ActionEvent e){
 String input = tf.getText();
   result = " ";
//Tokenizing the String
StringTokenizer st = new StringTokenizer(input);
while(st.hasMoreTokens()){
```

```
result += st.nextToken() + "\n";
}
repaint();
}
public void paint(Graphics g){
    g.drawString("Tokens:",20,120);
    int y=140;
    for(String line: result.split("\n")){
      g.drawstring(line,20,y);
        y+=20;
      }
    }
}
```

```
<html>
<body>
    <applet code="TokenizerApplet.class" width="400"
    height="300"></applet>
</body>
</html>
```

Output

Enter a sentence: my name is sahana

Tokenize

Tokens:

my

name

is

sahana

Lab Program - 19

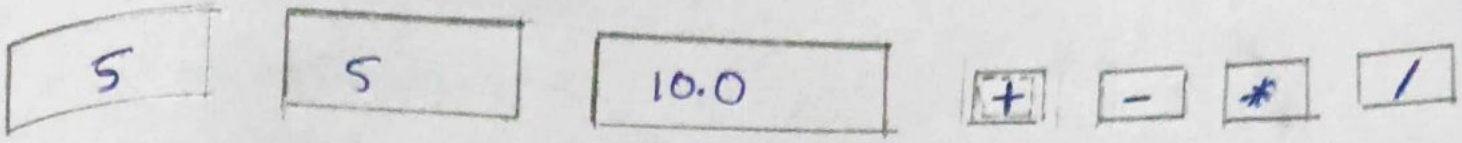
(19) Design a simple calculator using java applets

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class calculator extends Applet implements
ActionListener{

TextField t1 = new TextField(5);
TextField t2 = new TextField(5);
TextField result = new TextField(10);

Button add = new Button("+");
Button sub = new Button("-");
Button mul = new Button("*");
Button div = new Button("/");

public void init(){
 add(t1);
 add(t2);
 add(result);
 add(add);
 add(sub);
 add(mul);
 add(div);
add.addActionListener(this);
sub.addActionListener(this);
mul.addActionListener(this);
div.addActionListener(this);
}
```

```
public void actionPerformed (ActionEvent e){
    double a = Double.parseDouble(t1.getText());
    double b = Double.parseDouble(t2.getText());
    double r = 0;

    if (e.getSource() == add) r = a+b;
    if (e.getSource() == sub) r = a-b;
    if (e.getSource() == mul) r = a*b;
    if (e.getSource() == div) r = a/b;

     result.setText("" +r);

     }
     }
```

```
<html>
<body>
<applet code = "calculator.class" width = "300"
 height = "150"> </applet>
</body>
</html>
```

Output



Lab Program -20.

(20) Implement static and dynamic stack using interface using abstract class

```
import java.applet.Applet;
import java.awt.Graphics;

//-- Stack Interface --
interface stack{
   void push(int x);
   int pop();
   }

//-- Abstract stack --
abstract class BaseStack implements stack{
 int arr[];
 int top = -1;

 BaseStack(int size){
   arr = new int[size];
   }
  }
//-- static stack (fixed size) --
class staticStack extends BaseStack{
staticStack (int size){
  super(size);
  }
public void push(int x){
    if (top == arr.length-1)
      return; // no space
```

```
arr[++top] = x;
}
public int pop() {
    if (top == -1)
      return -1; // empty
    return arr[top--];
    }
    }

//--- Dynamic Stack (grows automatically) ---
class DynamicStack extends BaseStack {
 DynamicStack(int size){
    super(size);
    }
 public void push(int x){
    if (top == arr.length-1){
     // grow
     int temp[] = new int[arr.length*2];
     for(int i=0; i<arr.length; i++)
      temp[i] = arr[i];
      arr = temp;
       }
    arr[++top] = x;
    }
    public int pop(){
       if(top == -1)
          return -1;
        return arr[top--];
```

```
}
}

//--- Applet ---
public class StackApplet extends Applet{
static Stack s;
DynamicStack d;
public void init(){
    s = new StaticStack(3);
    d = new DynamicStack(2);
    s.push(5);
    d.push(15);
    d.push(25); // grows
    }
public void paint(Graphics g){
    g.drawString("static pop:" + s.pop(), 20, 40);
    g.drawString("Dynamic pop:" + d.pop(), 20, 60);
    }
}
</html>
<body>
<applet code="StackApplet.class" width="300"
    height="150">
</applet>
</body>
</html>
```

Output

static pop: 30

Dynamic pop: 25

Seen
21/12/25